OPMIG-99: Best Practice Summary

> **Series:** OPMIG — OpenPipeline Migration | **Notebook:** 10 of 10 | **Created:** March 2026 | **Last Updated:** 10/12/2026

Definitive best practice settings for migrating from classic logs to OpenPipeline. Each entry specifies the exact configuration.

Table of Contents

1. Migration Planning
2. Pipeline Configuration
3. Routing Rules
4. Bucket Strategy & Cost
5. Processing & Parsing
6. Metric Extraction (RED)
7. Security & Masking
8. Compliance
9. Troubleshooting & Validation
10. Data Limits & Constraints
11. DQL Cookbook

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail and OpenPipeline access — Managed is not covered by this series
Knowledge	Completion of or familiarity with OPMIG-01 through OPMIG-09
Purpose	This notebook serves as a reference card — no active environment required

1. Migration Planning

Practice	Recommended Setting/Value	Priority
Inventory all log sources first	`fetch logs, from:-7d | summarize count(), by:{log.source} | sort count desc`	Critical
Score sources with weighted priority matrix	Volume 25%, Cost Savings 25%, Security Risk 25%, Parsing Complexity 10%, Business Criticality 15%	Critical
Execute migration in 4 waves	Wave 1 (weeks 1-2): critical/security. Wave 2 (3-4): high-volume. Wave 3 (5-6): standard prod. Wave 4 (7+): dev/test	Critical
Keep same API endpoint	`/api/v2/logs/ingest` works identically for Classic and OpenPipeline — no code changes	Recommended
Maintain `logs.ingest` token scope	Add `metrics.ingest` only if using metric extraction	Critical
Backup pipeline config before every change	Settings API: `GET /api/v2/settings/objects?schemaIds=builtin:openpipeline.logs.pipelines` (and `builtin:openpipeline.logs.routing`) — store timestamped JSON backups	Critical

2. Pipeline Configuration

Practice	Recommended Setting/Value	Priority
One pipeline per use case	Name as `{source}-{purpose}` (e.g., `nginx-access-logs`)	Critical
Max 100 pipelines per scope	Consolidate with conditional processors if approaching limit	Recommended
Max 1,000 processors per pipeline (100 in a base pipeline)	Split into multiple pipelines if exceeded	Recommended
Max 10 DQL commands per processor (not on the limits page — verify)	Break into multiple processors if exceeded	Recommended
Processor order (Processing stage)	Masking → Drop → Parse → Enrich → Transform; bucket assignment and extraction are later, fixed stages	Critical
Name processors descriptively	`{action}-{target}` format (e.g., `mask-credit-cards`, `drop-debug-logs`)	Recommended
Test every processor with sample data	Use OpenPipeline UI "Sample data" field before saving	Critical

Practice	Recommended Setting/Value	Priority
Watch the default route	Unmatched records take the default route — the classic pipeline (`logs:default`) for logs and business events, the view-only built-in default pipeline elsewhere. High volume there = missing routes	Recommended
Max 5 pipelines extract per record	A record takes one route; a pipeline group adds its base pipelines. After 5, extraction stops but the record still persists	Recommended

> **Sources:** OpenPipeline limits (DT docs) — per-configuration-scope and per-pipeline tables; *"You can extract data on a single record in a maximum of five different pipelines"*.

3. Routing Rules

Practice	Recommended Setting/Value	Priority
Specific routes before general routes	First match wins — compliance routes at highest priority	Critical
Max 100 routes per scope	Combine conditions with AND/OR to stay within limit	Recommended
Max 10 conditions per route (not on the limits page — verify)	Use broader patterns if exceeded	Recommended
Route by `log.source` for source-based routing	`log.source == "nginx"` for specific apps	Recommended
Route by `k8s.namespace.name` for environments	`prod` → 35-day bucket, `staging` → 14-day, `dev` → 7-day	Recommended
Do NOT route on post-Processing entity fields	`dt.entity.service` , the Kubernetes and cloud-application entity fields and `dt.source_entity` are added AFTER the Processing stage (full list on the limits page, OPMIG-02)	Critical
Monitor unrouted log volume	`filter in(dt.openpipeline.pipelines, "logs:default")` grouped by `log.source` — classic-pipeline records; `isNull()` never fires because classic records carry the field too	Critical
A record takes the first matching route, and Bucket assignment is first-match-only	Order routes and bucket-assignment processors most specific first, so compliance data reaches its high-retention bucket	Critical

4. Bucket Strategy & Cost

Practice	Recommended Setting/Value	Priority
3-tier bucket strategy (small/medium orgs)	`critical_logs` 90d (10-15%), `default_logs` 35d (60-70%), `ephemeral_logs` 7d (20-30%)	Critical
5-tier bucket strategy (enterprise)	Add `compliance_logs` 365d (3-5%) and `security_logs` 180d (2-5%)	Critical
Bucket naming	`<environment>_<purpose>_logs` — max 100 chars, alphanumeric + underscore	Recommended
Drop DEBUG/TRACE before storage	Drop processor: `loglevel == "DEBUG" OR loglevel == "TRACE"` — reduction equals your DEBUG/TRACE share; measure it first (OPMIG-03)	Critical
Drop health check logs	`matchesValue(content, "*/health*") OR matchesValue(content, "*/ready*") OR matchesValue(content, "*/metrics*")` — measure the share first (`contains()` is not enabled in matchers)	Recommended
Extract metrics, then don't store the raw logs	Assign the raw logs **No storage assignment** — a Drop record processor runs before Metric extraction and would leave nothing to extract. 1M requests/day as logs = ~$140/mo; as metrics = ~$1/mo at placeholder rates (99.3% savings)	Recommended
Staging/dev short retention	Staging: 14 days. Dev: 7 days. Storage for that data falls with retention (7 of 35 days ≈ 80% less)	Recommended
Review bucket strategy quarterly	Usage trends, retention validation, unused buckets, compliance audit	Recommended

5. Processing & Parsing

Practice	Recommended Setting/Value	Priority
Use a Technology bundle where one exists	Apache HTTP, Nginx, Syslog (RFC 3164/5424), Java (Log4j, Logback, …) bundles; there is no generic JSON bundle — use `jsonExtract` for JSON logs	Recommended
Apache/Nginx DPL pattern	`IPADDR:client_ip SPACE '-' SPACE LD:user SPACE '[' TIMESTAMP(...):log_time ']' SPACE '"' LD:method SPACE LD:request_path SPACE LD:protocol '"' SPACE INT:status_code SPACE INT:response_bytes`	Recommended
Java/Log4j DPL pattern	`TIMESTAMP('yyyy-MM-dd HH:mm:ss,SSS'):log_timestamp SPACE '[' LD:thread ']' SPACE LD:level SPACE LD:logger SPACE '-' SPACE DATA:message`	Recommended

Practice	Recommended Setting/Value	Priority
Start key=value patterns with `LD?`	`parse` anchors at the start of the field: `LD? ('user='|'userId='|'user_id=') NSPACE:user_id` finds the key anywhere; without `LD?` it returns null unless the line begins with the key	Critical
Grok-to-DPL conversion	`%{IP}` → `IPADDR`, `%{INT}` → `INT`, `%{WORD}` → `WORD`, `%{GREEDYDATA}` → `DATA`	Recommended
Test patterns in DPL Architect	`https://{env}.apps.dynatrace.com/ui/apps/dynatrace.dpl.architect`	Critical
Target >90% parsing success rate	Query `countIf(isNotNull(loglevel))/count()` per pipeline — investigate below 90%	Recommended
Normalize missing log levels	`if(contains(content, "[ERROR]"), "ERROR", else: ...)` for logs without `loglevel`	Recommended

6. Metric Extraction (RED)

Practice	Recommended Setting/Value	Priority
Rate metric	Counter: `log.api.request_rate` with dimensions `method`, `path`, `status_category`, `service.name`	Recommended
Error metric	Counter: `log.api.request_errors` matching `status >= 500 OR loglevel == "ERROR"`	Recommended
Duration metric	Value: `log.api.response_time_ms` with dimensions `service.name`, `endpoint`, `status_category`	Recommended
Metric naming convention	`<source>.<domain>.<metric>.<unit>` (e.g., `log.api.response_time_ms`)	Recommended
Max 5-7 dimensions per metric	Never use `user_id`, `request_id`, `session_id`, `client_ip` as dimensions	Critical
Keep cardinality below 10,000 series	Bucket high-cardinality fields (e.g., duration → fast/normal/slow)	Critical
Normalize API paths in dimensions	`/api/users/12345` → `/api/users/{id}`	Recommended
Max 10 metric extractions per pipeline (not on the limits page — verify)	Max 5 event extractions, max 3 business event extractions	Recommended

7. Security & Masking

Practice	Recommended Setting/Value	Priority
Masking processors FIRST in the Processing stage	Sensitive data redacted before parsing, extraction and storage. Routing runs before any processor — keep PII out of routing conditions	Critical
Credit cards	Built-in `CREDITCARD` matcher, bounded: `!<<[0-9A-Za-z-] CREDITCARD !>>[0-9A-Za-z-]` → `[CC_REDACTED]`. Unbounded, it rewrites digit runs inside UUIDs	Critical
CVV codes	`('cvv='|'cvc=') [0-9]{3,4}` → `cvv=[REDACTED]`	Critical
Email addresses	`[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-9.-]+` → `[EMAIL_REDACTED]` (no built-in `EMAIL` matcher exists)	Critical
IP addresses (GDPR)	Built-in `IPADDR` matcher → `[IP_REDACTED]`	Critical
SSNs	`[0-9]{3} '-' [0-9]{2} '-' [0-9]{4}` → `[SSN_REDACTED]` (`INT` takes no `{n}` quantifier)	Critical
Bearer tokens	`'Bearer ' NSPACE` → `Bearer [TOKEN_REDACTED]`	Critical
API keys and tokens	`<<(('api_key' | 'API_KEY' | 'apiKey' | 'apikey' | 'APIKEY' | 'token' | 'TOKEN') '=') NSPACE` → `[REDACTED]`, plus a separate statement for JSON-quoted keys. DPL literals are case-sensitive; a trailing `LD` would erase the rest of the line	Critical
Passwords	`<<(('password' | 'PASSWORD' | 'Password' | 'passwd' | 'pwd') '=') NSPACE` → `[REDACTED]`; in URLs `'password=' [^&\s]+`	Critical
Remove always-sensitive fields	`fieldsRemove password, secret, api_key, token, authorization`	Recommended
Use descriptive placeholders	`[CC_REDACTED]`, `[EMAIL_REDACTED]`, `[PHI_REDACTED]` — enables audit counts by type	Recommended
Chain all masking in single processor	Apply CC, CVV, email, IP, SSN, tokens in sequence — single pass	Recommended
Validate masking weekly	`filter replacePattern(content, "!<<[0-9A-Za-z-] CREDITCARD !>>[0-9A-Za-z-]", replacement: "") != content` must return 0 — `matchesPhrase(content, "4111")` misses unbroken 16-digit numbers	Critical
Handle masking failure as critical incident	Stop storing the exposed records, identify exposure window, restrict access via IAM, delete exposed records (Grail record deletion API)	Critical

8. Compliance

Practice	Recommended Setting/Value	Priority
PCI-DSS	Bucket `pci_payment_logs`, 365+ days (Requirement 10.5.1: at least 12 months), mask PANs/CVV, read access by IAM policy for payment team + security + auditors	Critical
SOC 2	Bucket `soc2_audit_logs`, 365 days (no fixed SOC 2 period — match your audit window), security + auditors only. Buckets have no immutability setting	Critical
HIPAA	Bucket `hipaa_phi_logs`, 2555 days (7 years), mask patient IDs/SSNs/DOB/MRNs	Critical
GDPR	Bucket `eu_customer_logs`, 90 days max, mask emails/IPs, support erasure requests	Critical
Compliance routes at highest priority	Regulatory data must never accidentally route to under-retained buckets	Critical
Annual compliance audit	Verify retention, masking, erasure procedures, encryption, access controls	Recommended

9. Troubleshooting & Validation

Practice	Recommended Setting/Value	Priority
Post-migration validation checklist	All sources flowing, volumes match, no data loss, timestamps correct, routing correct, masking working, metrics generating	Critical
Monitor pipeline volume hourly	`makeTimeseries count(), by:{dt.openpipeline.pipelines}, interval:1h` — drops = routing failures	Recommended
Check log source health	`summarize last_seen = max(timestamp), by:{log.source}` — sources silent >30 min need investigation	Recommended
Combine DPL alternatives for performance	One `replacePattern` with `('a'|'b'|'c')` instead of chained `replacePattern` calls — fewer passes per record (only when one replacement text fits all)	Recommended
Drop processors before parse processors	Dropped records are never parsed — the saving is your dropped share	Critical
Never drop ERROR/FATAL in production	Safeguard: `loglevel == "DEBUG" AND NOT (loglevel == "ERROR" OR loglevel == "FATAL")`	Critical
Test changes in staging first	Backup → deploy staging → validate → document → rollback plan → production	Critical

10. Data Limits & Constraints

Practice	Recommended Setting/Value	Priority
Max record size	Request payload max 10 MB per configuration scope; record max 16 MB after processing — exceeding = rejected/dropped	Critical
Timestamp window	Logs: 24h past (72h from SaaS 1.348 — pre-release, staged tenant rollout planned from 09/22/2026; verify before relying on it). Metrics: 1h past. Future timestamps >10 min are **adjusted** to ingest time + 10 min, not dropped (spans excepted)	Critical
Max fields per record	Field-count, nesting and array limits are not on the limits page — verify in your tenant; 32 KB per log attribute	Recommended
Rate limits	Log Ingest API: 500 req/min/token. OTLP: 1,000 req/min. Config API: 100 req/min (not on the limits page — verify)	Recommended

> **Sources:** OpenPipeline limits (DT docs) — *"If the timestamp is more than 10 minutes in the future, it's adjusted to the ingest server time plus 10 minutes."*; *"The request payload size maximum limit is 10 MB per configuration scope."*; What's new in SaaS 1.348 (DT docs) — *"The log ingestion pipeline now accepts log records with timestamps up to 72 hours in the past, extended from the previous 24-hour limit."*

---

11. DQL Cookbook

Reference queries consolidated from across the OPMIG series — copy / paste into your tenant for ad-hoc validation, monitoring, or troubleshooting. These were previously scattered through OPMIG-09 (Troubleshooting & Validation); they live here as a cookbook so the main notebook stays focused on teaching content.

11.1 Parsing Validation

Diagnostic queries to check whether your DQL/DPL parsers are working correctly. Run these after configuring or modifying parse processors.

```
// Parsing success rate by pipeline
fetch logs, from: now() - 24h
| summarize {
    total = count(),
    with_loglevel = countIf(isNotNull(loglevel)),
    without_loglevel = countIf(isNull(loglevel))
  }, by: {dt.openpipeline.pipelines}
| fieldsAdd parse_rate = round((toDouble(with_loglevel) / toDouble(total)) * 100, decimals: 1)
| sort parse_rate asc
```

```
// Log level distribution (verify expected values)
fetch logs, from: now() - 24h
| summarize {log_count = count()}, by: {loglevel}
| sort log_count desc
```

```
// Find log sources with low parsing rates
fetch logs, from: now() - 24h
| summarize {
    total = count(),
    parsed = countIf(isNotNull(loglevel))
  }, by: {log.source}
| fieldsAdd parse_rate = round((toDouble(parsed) / toDouble(total)) * 100, decimals: 1)
| filter parse_rate &lt; 90
| sort parse_rate asc
```

```
// Sample unparsed logs from problematic sources
// Replace 'your-source' with source from previous query
fetch logs, from: now() - 1h
| filter log.source == "your-source"
| filter isNull(loglevel)
| fields content
| limit 30
```

```
// Verify custom field extraction
// Add your expected custom fields here
fetch logs, from: now() - 24h
| summarize {
    total = count(),
    with_request_id = countIf(isNotNull(request_id)),
    with_user_id = countIf(isNotNull(user_id)),
    with_order_id = countIf(isNotNull(order_id))
  }, by: {dt.openpipeline.pipelines}
```

11.2 Volume & Cost Validation

Verify post-migration data volumes match expectations and that drop processors are reducing volume as designed. Useful in the first week after cutover and as a recurring check.

```
// Daily log volume by bucket (cost impact)
fetch logs, from: now() - 7d
| fieldsAdd day = formatTimestamp(timestamp, format: "yyyy-MM-dd")
| summarize {log_count = count()}, by: {day, dt.system.bucket}
| sort day desc, log_count desc
```

```
// Volume trend - verify stable after migration
fetch logs, from: now() - 7d
| makeTimeseries {log_count = count()}, interval: 1h
```

```
// Check if debug/trace logs are being dropped (cost savings)
fetch logs, from: now() - 24h
| filter loglevel == "DEBUG" OR loglevel == "TRACE"
| summarize {debug_trace_count = count()}
| fieldsAdd status = if(debug_trace_count == 0,
    "✅ Debug/trace logs dropped as expected",
    else: "⚠️ Debug/trace logs still present - check drop processors")
```

```
// Volume by log level (should see mostly INFO/WARN/ERROR)
// percentage = share of all logs in the window, computed against the overall total
fetch logs, from: now() - 24h
| summarize {log_count = count()}, by: {loglevel}
| summarize {total = sum(log_count), rows = collectArray(record(loglevel = loglevel, log_count
= log_count))}
| expand rows
| fieldsAdd loglevel = rows[loglevel], log_count = rows[log_count]
| fieldsAdd percentage = round(toDouble(log_count) / toDouble(total) * 100, decimals: 1)
| fields loglevel, log_count, percentage
| sort log_count desc
```

```
// Top 10 highest volume sources
fetch logs, from: now() - 24h
| summarize {log_count = count()}, by: {log.source}
| sort log_count desc
| limit 10
```

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.